



INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE

EXTRACCIÓN DEL CONOCIMIENTO EN BASES DE DATOS

T4. Optimización de modelos

PRESENTA:

BERTADILLO VILLALOBOS LEOBARDO DANIEL

GRUPO: 9º C

Profesora:

VIRNA VIRIDIANA VELA RINCÓN

Emiliano Zapata, Morelos, México.

Resumen

Este reporte investiga en qué consiste la **optimización de modelos supervisados** y la aplica a tres clasificadores (KNN, SVM y Árbol de Decisión) sobre el conjunto de datos Iris. La idea es partir de los modelos con sus valores por defecto (la **línea base**) y mejorarlos ajustando sus **hiperparámetros** mediante búsqueda en rejilla con validación cruzada, para luego comparar cómo cambia su desempeño. Los resultados muestran que la optimización mejora claramente a KNN (de **91.11 %** a **95.56 %** de exactitud) y, sobre todo, al Árbol de Decisión (de **91.11 %** a **97.78 %**) al limitar su profundidad para evitar el sobreajuste; SVM, que ya partía de un buen desempeño, se mantiene en **93.33 %**. Se concluye que la optimización es muy útil, aunque su efecto depende del modelo y de qué tan ajustada estaba ya su configuración inicial.

Palabras clave: optimización de modelos, hiperparámetros, búsqueda en rejilla, validación cruzada, sobreajuste, KNN, SVM, árbol de decisión.

Índice

1. Introducción	1
2. Investigación	1
2.1. ¿Qué es la optimización de modelos supervisados?	1
2.2. El problema del sobreajuste	1
2.3. Validación cruzada	1
2.4. Métodos de búsqueda de hiperparámetros	2
3. Metodología	2
4. Resultados y comparación	3
4.1. Efecto en la validación cruzada	3
4.2. Desempeño en el conjunto de prueba	3
4.3. Matrices de confusión	4
5. Conclusiones	6
Bibliografía	7

1. Introducción

La clasificación de flores del conjunto Iris a partir de las medidas de sus pétalos y sépalos es un problema clásico para comparar algoritmos de aprendizaje supervisado. Un modelo, sin embargo, casi nunca da su mejor resultado con la configuración inicial: la mayoría de los algoritmos tienen “perillas” que se pueden ajustar para que aprendan mejor. A ese proceso de ajuste se le llama **optimización de modelos**.

2. Investigación

2.1. ¿Qué es la optimización de modelos supervisados?

En el aprendizaje supervisado un modelo aprende a partir de ejemplos ya etiquetados para luego predecir la etiqueta de casos nuevos. **Optimizar** un modelo significa buscar la configuración con la que obtiene su mejor desempeño posible, sin memorizar los datos de entrenamiento ni fallar con datos nuevos [Hastie et al., 2009, Géron, 2019]. En la práctica, esto consiste sobre todo en elegir bien sus hiperparámetros y en validar correctamente esa elección.

Definición 2.1: Parámetros e hiperparámetros

Los **parámetros** son los valores que el modelo aprende solo durante el entrenamiento (por ejemplo, los cortes internos de un árbol). Los **hiperparámetros**, en cambio, se fijan antes de entrenar y controlan cómo aprende el modelo: el número de vecinos k en KNN, los valores de C y γ en SVM, o la profundidad máxima de un árbol. La optimización trabaja justamente sobre estos últimos [Probst et al., 2019].

2.2. El problema del sobreajuste

La meta de un modelo no es acertar en los datos que ya conoce, sino **generalizar** a datos nuevos. Si un modelo se ajusta demasiado a los datos de entrenamiento, aprende hasta el ruido y falla con casos nuevos: a esto se le llama **sobreajuste** (“overfitting”). El caso contrario, un modelo demasiado simple que no capta los patrones, se llama **subajuste** (“underfitting”) [James et al., 2013]. Un ejemplo claro es el árbol de decisión: si se le deja crecer sin límite, memoriza el entrenamiento; limitar su profundidad lo obliga a quedarse con las reglas más generales [Géron, 2019]. Una buena optimización busca el punto medio entre ambos extremos.

2.3. Validación cruzada

Para elegir hiperparámetros sin hacer trampa no se deben usar los datos de prueba. La técnica más común es la **validación cruzada de k particiones** (“k-fold cross-validation”): el conjunto de entrenamiento se divide en k partes, se entrena con $k - 1$ y se evalúa con la restante, repitiendo el proceso k veces y promediando el resultado [James et al., 2013]. Así se obtiene una estimación más confiable del desempeño y se reduce el riesgo de elegir una configuración por suerte.

2.4. Métodos de búsqueda de hiperparámetros

Existen varias formas de explorar las combinaciones posibles de hiperparámetros [Bergstra and Bengio, 2012, Géron, 2019]:

- **Búsqueda en rejilla (“Grid Search”).** Se define una lista de valores para cada hiperparámetro y se prueban todas las combinaciones. Es exhaustiva y sencilla, pero su costo crece rápido al aumentar los parámetros.
- **Búsqueda aleatoria (“Random Search”).** Se prueban combinaciones elegidas al azar. Suele encontrar buenas configuraciones con menos pruebas, sobre todo cuando hay muchos hiperparámetros [Bergstra and Bengio, 2012].
- **Optimización bayesiana.** Usa los resultados ya obtenidos para decidir de forma inteligente qué combinación probar a continuación, siendo más eficiente en problemas grandes.

Se emplea **búsqueda en rejilla con validación cruzada**, por ser clara y adecuada al tamaño del problema.

El papel del preprocesamiento

Optimizar no es solo ajustar hiperparámetros. Un paso previo igual de importante es preparar los datos: por ejemplo, **estandarizar** las variables para que todas estén en una escala comparable. KNN y SVM se basan en distancias, así que sin este paso su desempeño cae [Géron, 2019]. El árbol de decisión es una excepción: no le afecta la escala, porque decide con cortes sobre cada variable por separado. Aun así, en este experimento se usa el mismo preprocesamiento para los tres modelos, de modo que la comparación refleje solo el efecto de optimizar los hiperparámetros.

3. Metodología

Se utilizó el conjunto Iris [Dua and Graff, 2019], con **150 ejemplos**, **4 medidas** (largo y ancho de sépalos y pétalos) y **3 especies** (setosa, versicolor y virginica) con 50 ejemplos cada una. El experimento, implementado en Python con `scikit-learn` [Pedregosa et al., 2011], siguió estos pasos:

1. **Preparación.** Se dividieron los datos en 70 % para entrenar y 30 % para probar (45 flores de prueba), manteniendo la proporción de clases, y se estandarizaron las variables (la escala se calculó solo con el entrenamiento).
2. **Línea base.** Se entrenaron los tres modelos con sus valores **por defecto**: KNN con $k = 5$; SVM con kernel RBF, $C = 1$ y γ automático; y un Árbol de Decisión sin límite de profundidad.
3. **Optimización.** Con **búsqueda en rejilla** y validación cruzada de 5 particiones se buscaron los mejores hiperparámetros de cada modelo (Tabla 1).
4. **Comparación.** Ambas versiones (base y optimizada) se evaluaron sobre el mismo conjunto de prueba, midiendo exactitud, precisión, sensibilidad, especificidad y Macro F1.

Tabla 1: Hiperparámetros explorados en la búsqueda en rejilla.

Modelo	Hiperparámetros probados
KNN	número de vecinos (1-20), tipo de peso, métrica de distancia
SVM	kernel (lineal, RBF, polinomial), C , γ
Árbol	criterio (Gini/entropía), profundidad máxima, mínimos para dividir y por hoja

La búsqueda seleccionó: **KNN** con $k = 14$ y distancia euclidiana; **SVM** con kernel lineal y $C = 100$; y **Árbol de Decisión** con criterio Gini y profundidad máxima de 3.

4. Resultados y comparación

4.1. Efecto en la validación cruzada

La optimización mejoró la validación cruzada de los tres modelos, que es justamente lo que la búsqueda intenta maximizar (Tabla 2).

Tabla 2: Exactitud por validación cruzada (entrenamiento) antes y después de optimizar.

Modelo	Por defecto	Optimizado	Cambio
KNN	0.9524	0.9810	+0.0286
SVM	0.9714	0.9810	+0.0096
Árbol	0.9429	0.9524	+0.0095

4.2. Desempeño en el conjunto de prueba

La Tabla 3 reúne todas las métricas sobre los datos de prueba y la Figura 1 las resume visualmente.

Tabla 3: Comparación de métricas sobre el conjunto de prueba (45 instancias).

Modelo	Accuracy	Precisión	Sensibilidad	Especificidad	Macro F1
KNN base	0.9111	0.9298	0.9111	0.9556	0.9095
KNN optimizado	0.9556	0.9608	0.9556	0.9778	0.9554
SVM base	0.9333	0.9345	0.9333	0.9667	0.9333
SVM optimizado	0.9333	0.9444	0.9333	0.9667	0.9327
Árbol base	0.9111	0.9155	0.9111	0.9556	0.9107
Árbol optimizado	0.9778	0.9792	0.9778	0.9889	0.9778

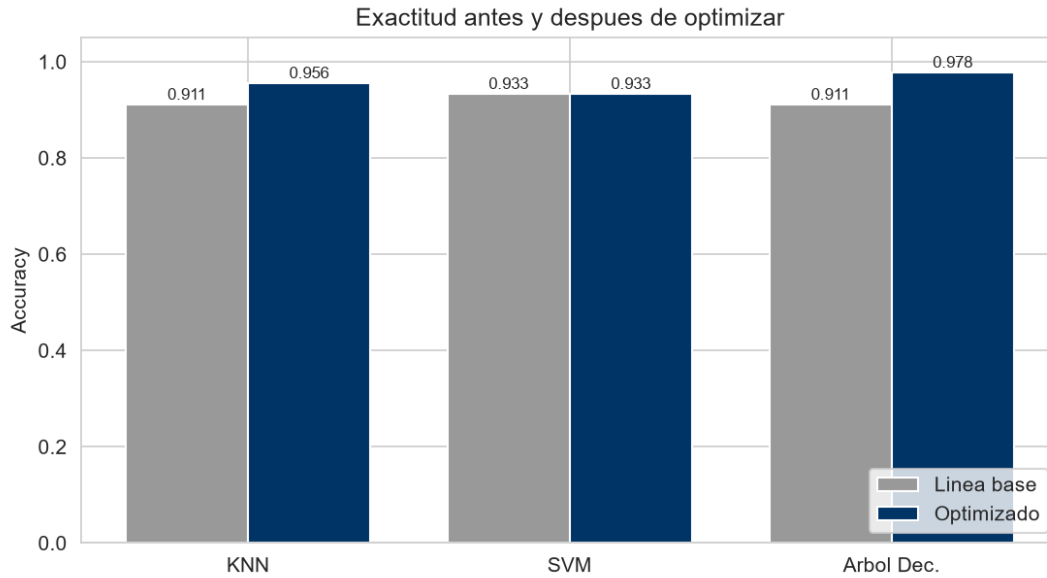
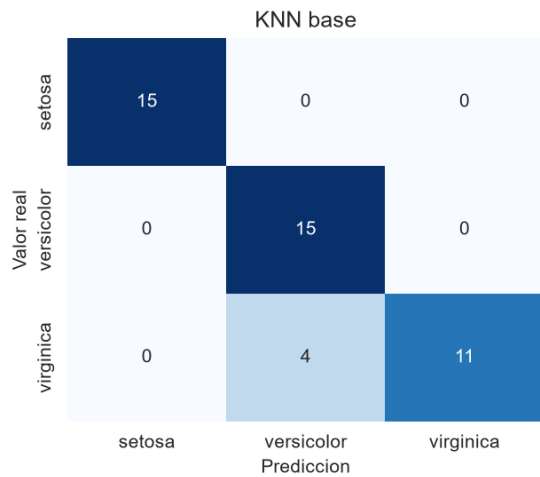


Figura 1: Exactitud de cada modelo antes y después de optimizar.

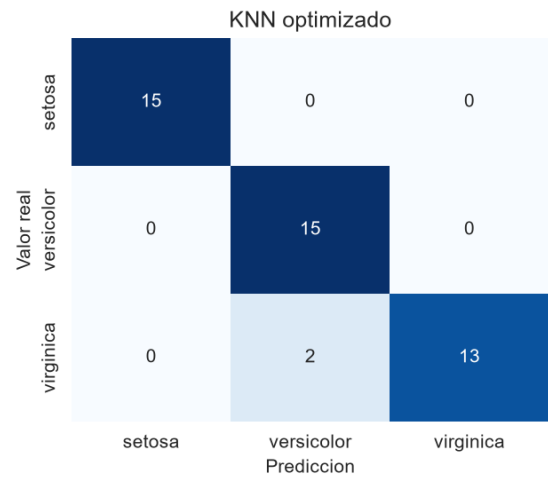
El **Árbol de Decisión** fue el que más mejoró: su exactitud subió de 91.11 % a 97.78 %. La razón es clara: por defecto el árbol crecía sin límite y se sobreajustaba; al fijar una profundidad máxima de 3 se quedó con las reglas más generales y acertó más en datos nuevos. **KNN** también mejoró de forma notable (de 91.11 % a 95.56 %) al usar más vecinos ($k = 14$ en lugar de 5), lo que suaviza su decisión. **SVM**, en cambio, ya partía de un desempeño alto y se mantuvo en 93.33 %: su validación cruzada mejoró un poco, pero en la prueba el resultado fue casi el mismo, señal de que su configuración inicial ya era buena para este problema.

4.3. Matrices de confusión

Las matrices de confusión muestran dónde acierta y dónde falla cada modelo. En las tres (Figuras 2, 3 y 4) la especie setosa se clasifica perfectamente, porque es muy distinta de las otras dos. Toda la dificultad está entre versicolor y virginica, que se parecen más. La optimización reduce esas confusiones en KNN y en el árbol; en SVM se mueven mínimamente.

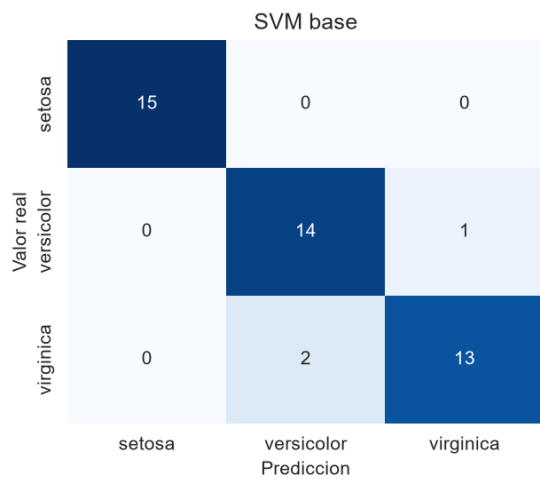


(a) KNN base

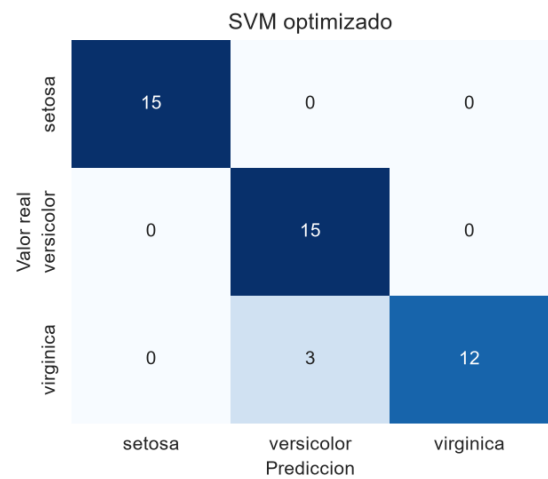


(b) KNN optimizado

Figura 2: Matrices de confusión de KNN antes y después de optimizar.



(a) SVM base



(b) SVM optimizado

Figura 3: Matrices de confusión de SVM antes y después de optimizar.

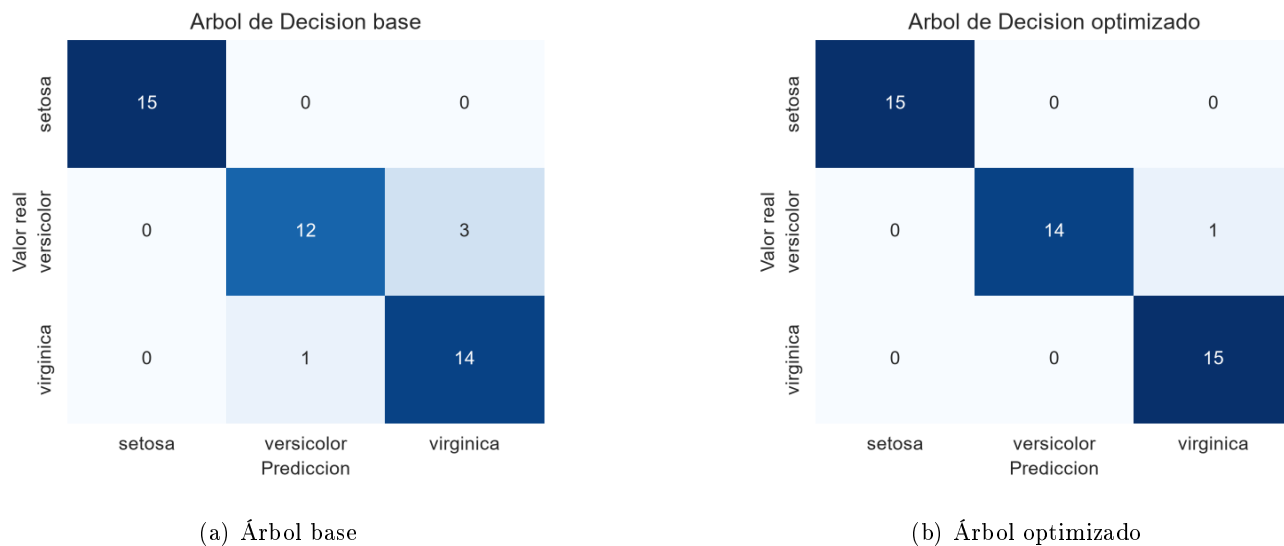


Figura 4: Matrices de confusión del Árbol de Decisión antes y después de optimizar.

5. Conclusiones

La investigación mostró que optimizar un modelo supervisado va más allá de entrenarlo: implica preparar bien los datos, elegir sus hiperparámetros con métodos como la búsqueda en rejilla y validar esa elección con validación cruzada para evitar el sobreajuste.

Al aplicarlo sobre Iris, los tres modelos mejoraron su validación cruzada, pero el efecto en la prueba fue distinto en cada caso. El **Árbol de Decisión** tuvo la mejora más grande (de 91.11 % a 97.78 %), un ejemplo muy claro de cómo limitar la complejidad de un modelo reduce el sobreajuste. **KNN** también mejoró de forma apreciable, mientras que **SVM** prácticamente se mantuvo porque ya partía de una buena configuración.

La principal lección es que la optimización **sí ayuda, pero su impacto depende del modelo y de qué tan ajustada estaba su configuración inicial**: un modelo que ya funciona bien deja poco margen de mejora, mientras que uno mal configurado (como el árbol sin podar) puede ganar mucho. Por eso, más importante que optimizar a ciegas es medir, comparar y entender por qué cambia el desempeño de cada modelo.

Referencias

- [Bergstra and Bengio, 2012] Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305.
- [Dua and Graff, 2019] Dua, D. and Graff, C. (2019). UCI machine learning repository. University of California, Irvine, School of Information and Computer Sciences. Consultado el 29 de junio de 2026.
- [Géron, 2019] Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*. O’Reilly Media, Sebastopol, CA, 2 edition.
- [Hastie et al., 2009] Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference and Prediction*. Springer, New York, 2 edition.
- [James et al., 2013] James, G., Witten, D., Hastie, T., and Tibshirani, R. (2013). *An Introduction to Statistical Learning with Applications in R*. Springer, New York.
- [Pedregosa et al., 2011] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.
- [Probst et al., 2019] Probst, P., Boulesteix, A.-L., and Bischl, B. (2019). Tunability: Importance of hyperparameters of machine learning algorithms. *Journal of Machine Learning Research*, 20(53):1–32.